PLN · PC.8 · GRUPO G.4

PC.8 de Cabeça

Tudo que precisa estar na cabeça pra apresentar o chatbot de FAQ escolar sem consultar nada. Leia na ordem uma vez. Depois treine as perguntas da parte 6 falando em voz alta.

Todos os números deste guia foram tirados rodando o projeto de verdade (stemmer ligado).

PARTE 1

O projeto em 30 segundos

Se for sorteado, comece com isto. Decore a ideia, não precisa ser palavra por palavra.

FALE ASSIM

"Fizemos um chatbot que responde dúvidas de uma escola sobre 5 assuntos. Quando alguém pergunta, ele limpa o texto, transforma em números com bag of words e TF-IDF, compara com as 77 perguntas que já conhece usando similaridade de cosseno, e as 5 mais parecidas votam na resposta. Se nada for parecido o suficiente, ele admite que não sabe e mostra os assuntos que conhece. Tudo fica salvo num banco SQLite e roda numa página web feita em Flask."

PARTE 2

A ideia inteira em 5 verbos

O chatbot faz sempre estas 5 coisas, nessa ordem. Se você lembrar dos verbos, consegue explicar todo o resto puxando um por um.

LIMPA → CONTA → COMPARA → VOTA → (OU DESISTE)

LIMPA	CONTA	COMPARA	VOTA	DESISTE
deixa o texto padronizado	vira lista de números	mede o quão parecido é	as 5 mais parecidas decidem	se nada for parecido
pré-processamento	bag of words + TF-IDF	cosseno	votação por maioria	limiar 0,15 / fallback

Vamos seguir uma pergunta só do começo ao fim: **"Qual é o Horário da Biblioteca?"**

VERBO 1

LIMPA: pré-processamento

O computador acha que "Horário", "horário" e "horario" são três palavras diferentes. Então antes de tudo a gente padroniza o texto. São 6 etapas, sempre na mesma ordem, aplicadas tanto nas perguntas de treino quanto na pergunta do usuário.

Início	`Qual é o Horário da Biblioteca?`
1. Minúsculas H = h	`qual é o horário da biblioteca?`
2. Acentos quem digita sem acento	`qual e o horario da biblioteca?`
3. Pontuação "aula?" = "aula"	`qual e o horario da biblioteca`
4. Tokenização quebra em palavras	`[qual, e, o, horario, da, biblioteca]`
5. Stopwords tira palavra vazia	`[horario, biblioteca]`
6. Stemização corta o final	`[horari, bibliotec]`

Pra decorar a ordem: **M**eu **A**migo **P**edro **T**oma **S**orvete **S**empre. Minúsculas, Acentos, Pontuação, Tokenização, Stopwords, Stem.

- **Stopword** é palavra que aparece em tudo e não diz o assunto: "o", "da", "qual", "de". A lista está escrita dentro do código, então não precisa de internet.
- **Stemização** corta o final da palavra pra juntar a família: "matricular", "matrícula" e "matriculado" viram todos `matricul`. Usamos o stemmer **RSLP**, que é feito pro português (vem do NLTK).
- **Stem x lema:** o stem só corta por regra e pode gerar algo que não é palavra (`horari`). A lematização procura a palavra de dicionário ("matricular"), mas precisa de um modelo de língua pesado, como o spaCy. Pra comparar frases, cortar já basta.

VERBO 2

CONTA: bag of words e TF-IDF

Bag of words ("saco de palavras") é uma lista de presença. Juntando todas as palavras das 77 perguntas de treino (depois de limpar), sobram 96 palavras diferentes: esse é o **vocabulário**. Toda frase vira uma fila de 96 números, um pra cada palavra do vocabulário. Quase todos ficam zero.

É um saco mesmo: você joga as palavras dentro e perde a ordem. "biblioteca horário" e "horário biblioteca" viram o mesmo vetor.

- **Binário:** 1 se a palavra aparece, 0 se não.
- **Por contagem:** quantas vezes ela aparece.

O problema: assim toda palavra vale igual. Mas uma palavra que aparece em quase todas as perguntas não ajuda a descobrir o assunto.

TF-IDF resolve isso: palavra rara vale mais.

- **TF** (frequência do termo): quantas vezes a palavra aparece nesta frase.
- **IDF** (frequência inversa nos documentos): quanto mais frases do treino usam a palavra, menor o peso dela.

Numa sala em que todo mundo usa uniforme, "está de uniforme" não ajuda a achar ninguém. "Está de boné vermelho" ajuda. A palavra rara é o boné vermelho.

PALAVRA (JÁ CORTADA)	APARECE EM	PESO IDF
`secret` (secretaria)	15 de 77 frases	2,58 (o menor)
`horari` (horário)	9 de 77	3,05
`bibliotec` (biblioteca)	7 de 77	3,28
`almoc` (almoço)	1 de 77	4,66 (o maior)

Fórmula do material do professor:

```
tfidf = tf × idf     idf = log( N / df )
```

N = total de frases (77). **df** = em quantas frases a palavra aparece. O scikit-learn usa uma versão "suavizada", com +1 pra nunca dividir por zero:

```
idf = ln( (1 + N) / (1 + df) ) + 1
```

Por último o vetor é **normalizado** (norma L2): ele é encolhido ou esticado até ter tamanho 1. Assim só importa *pra onde ele aponta* (quais palavras e em que proporção), não o tamanho da frase.

VERBO 3

COMPARA: similaridade de cosseno

Cada frase agora é uma seta (vetor). O cosseno mede o **ângulo** entre duas setas.

- `1` = apontam pro mesmo lado = mesmas palavras, mesmo peso.
- `0` = não têm nenhuma palavra em comum.
- Como não existe peso negativo, o resultado fica sempre entre `0` e 1.

```
cos = (A · B) / ( |A| × |B| )
```

Como o TF-IDF já deixou as setas com tamanho 1, a conta vira só a multiplicação ponto a ponto somada (produto escalar).

Conta de exemplo, sem pesos, com um vocabulário de 3 palavras [horari, bibliotec, aul]:

```
pergunta "horário da biblioteca" = [1, 1, 0]
treino   "horário das aulas"    = [1, 0, 1]
cos = (1·1 + 1·0 + 0·1) / (√2 × √2) = 1 / 2 = 0,5
```

Por que não distância normal (euclidiana)? Porque uma frase longa e uma curta sobre o mesmo assunto ficariam "longe" só por causa do tamanho. O cosseno olha a direção, então as duas continuam parecidas.

VERBO 4

VOTA: votação por maioria

O sistema calcula o cosseno da pergunta contra as 77 frases de treino e pega as 5 mais parecidas (K = 5). Cada uma dessas 5 pertence a uma "intenção" (uma resposta pronta) e vota nela. **Ganha a intenção com mais votos.**

Resultado real de "Qual o horário da biblioteca?":

#	FRASE DE TREINO PARECIDA	VOTA EM	COSSENO
1°	Qual o horário de atendimento da biblioteca?	horario_biblioteca	0,767
2°	Qual o horário de funcionamento da biblioteca?	horario_biblioteca	0,693
3°	Qual o horário das aulas?	horario_aulas	0,465
4°	Que horas a biblioteca abre?	horario_biblioteca	0,397
5°	Qual o horário de atendimento da secretaria?	horario_atendimento_secretaria	0,380

Placar: biblioteca 3, aulas 1, secretaria 1. Ganha biblioteca. A "confiança" mostrada na tela (0,767) é o maior cosseno entre os que votaram na vencedora.

- **Empate?** Ganha quem tem a maior soma de cossenos. Se ainda empatar, quem tem o maior cosseno sozinho.
- **Por que não pegar só a mais parecida?** Uma frase de treino mal escrita poderia decidir tudo sozinha. Com 5 votando, um erro isolado perde.
- Essa ideia tem nome: **KNN** (K vizinhos mais próximos), usando cosseno como medida.

É como perguntar o caminho pra 5 pessoas em vez de 1: se uma errar, as outras 4 corrigem.

VERBO 5

DESISTE: limiar e fallback

O chatbot não inventa resposta. Ele desiste em dois casos:

- a frase mais parecida tem cosseno menor que 0,15;
- nenhuma palavra da pergunta existe no vocabulário (o vetor fica todo zero). Palavra que nunca apareceu no treino é simplesmente ignorada.

Aí ele responde que não entendeu e lista os 5 assuntos que sabe. Isso é o **fallback**. Exemplo: "Qual a previsão do tempo?" dá cosseno 0,0, porque "previsão" e "tempo" não existem no vocabulário.

PARTE 3

Números pra saber de cabeça

5 categorias (assuntos)	**15** intenções (3 por categoria), cada uma com 1 resposta pronta	**77** frases de treino (jeitos diferentes de perguntar)	**96** palavras no vocabulário depois de limpar
K=5 frases que votam	**0,15** cosseno mínimo pra responder	**12/12** testes certos (100%), mais 2 perguntas fora do escopo que caem no fallback	**6** etapas de pré-processamento

As 5 categorias

Pra decorar: **H**oje **M**inha **B**ibi **S**aiu **N**ervosa.

CATEGORIA	AS 3 INTENÇÕES
Horários e Calendário Acadêmico	horário das aulas · horário das disciplinas · calendário e férias
Matrícula e Documentos	como se matricular · declaração de matrícula · trancamento
Biblioteca	horário de funcionamento · empréstimo de livro · renovação
Secretaria e Atendimento	telefone e e-mail · localização · horário de atendimento
Notas, Faltas e Avaliações	consultar notas · consultar faltas · recuperação

PARTE 4

Onde fica cada coisa no código

Cada arquivo cuida de um pedaço. Repare que os verbos batem com os arquivos.

ARQUIVO	O QUE FAZ	VERBO
`chatbot/preprocessing.py`	as 6 etapas de limpeza	LIMPA
`chatbot/engine.py`	TF-IDF (`TfidfVectorizer`), cosseno (`cosine_similarity`), votação na função `answer()` , fallback	CONTA, COMPARA, VOTA, DESISTE
`chatbot/database.py`	banco SQLite: categorias, intenções, frases de treino e histórico de conversas	guarda tudo
`app.py` + `templates/` + `static/`	site em Flask: a página do chat e a rota `/api/chat`	mostra
`data/faq_seed.json`	as perguntas e respostas do FAQ, editáveis sem mexer em código	conteúdo

O caminho de uma pergunta

Usuário digita na página → o navegador manda pra `/api/chat` → o Flask chama `engine.answer()` → limpa, vira vetor com o **mesmo** vocabulário do treino, compara, vota →

salva no histórico do SQLite (tabela `conversation_log`) → devolve a resposta pra tela.

O TF-IDF é "treinado" (`fit`) **uma vez só**, quando o `app.py` liga. Depois disso o vocabulário fica fixo.

PARTE 5

A história da pizza

Contem isso na apresentação. Mostra que o grupo testou, achou um erro, entendeu a causa e corrigiu.

O QUE ACONTECEU

Testamos "Quero pedir uma pizza". O chatbot respondeu sobre **trancamento de curso**, com confiança 0,48. Errado.

POR QUÊ

A palavra "quero" só aparecia numa frase de treino: "Quero trancar o curso". Palavra rara ganha IDF alto, então "quero" pesou muito, mesmo sem ter nada a ver com trancamento.

O QUE FIZEMOS

Colocamos "quero", "queria", "preciso", "desejo" na lista de stopwords. Agora a pizza cai no fallback, como deveria.

FALE ASSIM

"Isso mostra uma limitação do TF-IDF: ele dá peso alto pra palavra rara mesmo quando ela não tem significado nenhum pro assunto."

Bônus: outra limitação que dá pra mostrar

"horas" vira `hor` e "horário" vira `horari`. O stemmer não junta as duas. Por isso "Que horas a biblioteca abre?" ficou só com 0,397 na tabela da votação, mesmo sendo sobre a mesma coisa. O sistema compara letras, não significado.

PARTE 6

Perguntas que o professor pode fazer

Treino: leia a pergunta, responda em voz alta, só depois abra. A **resposta curta** é o que você fala. O **se ele pedir mais** é pra quando ele cavar.

No papel: cubra a resposta com a mão antes de ler.

01 **O que é bag of words?**

Transformar a frase numa lista de quais palavras do vocabulário aparecem nela, sem ligar pra ordem.

Se ele pedir mais: pode ser binário (0 ou 1) ou por contagem (quantas vezes). Nosso vocabulário tem 96 palavras, então cada frase vira um vetor de 96 números. Na prática usamos a versão com TF-IDF, que parte da contagem e depois dá peso.

02 **Por que TF-IDF e não só contar as palavras?**

Porque contar dá o mesmo peso pra toda palavra. O TF-IDF dá mais peso pra palavra rara, que é a que diferencia um assunto do outro.

Se ele pedir mais: "secretaria" aparece em 15 das 77 frases e tem peso 2,58. "almoço" aparece em 1 e tem peso 4,66.

03 **Qual a fórmula do IDF?**

idf = log(N / df). N é o total de frases do corpus e df é em quantas frases a palavra aparece. O TF-IDF é tf × idf.

Se ele pedir mais: o scikit-learn usa a versão suavizada, ln((1+N)/(1+df)) + 1. O +1 de dentro evita divisão por zero, o +1 de fora garante que nenhuma palavra fique com peso zero. Depois o vetor é normalizado pra ter tamanho 1 (norma L2).

04 **O que é similaridade de cosseno? Por que não distância euclidiana?**

É o cosseno do ângulo entre os dois vetores: 1 é mesma direção, 0 é nada em comum. Usamos porque importa quais palavras aparecem, não o tamanho da frase.

Se ele pedir mais: cos = (A·B)/(|A|·|B|). Como os vetores já têm tamanho 1, vira só o produto escalar. Frase longa e frase curta sobre o mesmo assunto ficam longe na euclidiana, mas com cosseno alto.

05 **Como funciona a votação por maioria?**

Pegamos as 5 frases de treino mais parecidas. Cada uma vota na resposta a que pertence. Ganha a mais votada.

Se ele pedir mais: no empate, ganha a maior soma de cossenos; se empatar de novo, o maior cosseno individual. Exemplo da biblioteca: 3 votos contra 1 e 1.

06 **Por que não responder só com a frase mais parecida (top-1)?**

Porque uma frase de treino ruim decidiria sozinha. Com 5 votando, um erro isolado perde. É a mesma ideia do KNN.

07 **E se perguntarem algo fora do assunto?**

Se o melhor cosseno for menor que 0,15, ou se nenhuma palavra for conhecida, ele diz que não entendeu e lista os 5 assuntos que sabe. Não inventa resposta.

08 **E uma palavra que nunca apareceu no treino?**

É ignorada. O vocabulário é montado uma vez quando o app liga, e palavra nova não tem posição no vetor.

Se ele pedir mais: é por isso que "Qual a previsão do tempo?" dá cosseno 0: nenhuma das palavras existe no vocabulário.

09 **Quais etapas de pré-processamento vocês fizeram?**

Seis, nessa ordem: minúsculas, remoção de acentos, remoção de pontuação, tokenização, remoção de stopwords e stemização com RSLP. (Meu Amigo Pedro Toma Sorvete Sempre.)

Se ele pedir mais: acento é importante porque muita gente digita "horario" sem acento. Sem essa etapa, "horário" e "horario" virariam palavras diferentes e o cosseno daria zero.

10 **Por que stemização e não lematização?**

Stemização corta o final por regra (matrícula → matricul). Lematização acha a palavra de dicionário, mas precisa de um modelo pesado como o spaCy. Pra juntar palavras da mesma família, cortar já basta.

11 **Que persistência vocês usaram e por quê?**

SQLite. É um banco que fica num arquivo só (`data/faq.db`) e não precisa instalar servidor.

Se ele pedir mais: guarda as categorias, intenções, frases de treino e o histórico de toda conversa (pergunta, resposta, categoria, confiança, data). Todo o acesso ao banco está em `database.py`, então dá pra trocar por PostgreSQL mexendo só nele.

12 **O que é o corpus e o vocabulário? Quando são montados?**

Corpus são as 77 frases de treino. Vocabulário são as 96 palavras diferentes que sobram depois de limpar. O TF-IDF é treinado (fit) uma vez quando o app liga.

13 **Como adiciono uma pergunta ou categoria nova?**

Edita o `data/faq_seed.json`, roda `python seed_db.py --force` e reinicia o `app.py`. Não mexe em código.

Se ele pedir mais: tem que reiniciar porque o TF-IDF só é treinado quando o app liga.

14 **Quais as limitações?**

Quatro: ignora a ordem das palavras, não entende sinônimo que não está no treino, palavra rara pode pesar demais, e não lembra da pergunta anterior.

Se ele pedir mais: sinônimo: se "docente" nunca apareceu, ele não sabe que é "professor". Palavra rara: o caso da pizza. Outro exemplo: "horas" e "horário" viram radicais diferentes (`hor` e `horari`).

15 **Como dava pra melhorar?**

Trocar o TF-IDF por embeddings, tipo Word2Vec. Neles, palavras de sentido parecido ficam perto, mesmo que nunca tenham aparecido juntas.

Se ele pedir mais: a frase seria representada pela média dos vetores das palavras dela, como na seção 6 do material do professor.

16 **De onde vem o número de "confiança" que aparece na tela?**

É o maior cosseno entre as frases que votaram na intenção vencedora. Na biblioteca deu 0,767.

PARTE 7

Demo ao vivo

Antes do dia, com internet, no computador da apresentação

Rodem a instalação completa. Se pular o `setup_nltk.py`, a stemização desliga **sem avisar** e o teste cai pra **83% (10/12)** na frente do professor. A gente testou isso.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python setup_nltk.py
```

Se o PowerShell bloquear a ativação: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

No dia, nesta ordem

1. Abrir o terminal na pasta do projeto e ativar o ambiente.

   ```
   .venv\Scripts\Activate.ps1
   ```

2. Rodar os testes. Mostrar: **100% (12/12)** e as 2 perguntas fora do escopo caindo no fallback.

   ```
   python tests/test_chatbot.py
   ```

3. Ligar o site e abrir `http://127.0.0.1:5000`.

   ```
   python app.py
   ```

4. Ler em voz alta as 5 categorias da barra lateral. Isso é o "contexto" que o enunciado pede.

5. Perguntar **"Qual o horário da biblioteca?"** e clicar em **"Mostrar detalhes técnicos"**. Apontar os 3 votos da biblioteca contra 1 e 1.

6. Perguntar **"Como faco minha matricula"** (sem acento, sem ç). Acerta com 1,0. Isso prova que a limpeza funciona.

7. Perguntar **"Qual a previsão do tempo?"** e mostrar o fallback com a lista de assuntos.

8	Contar a história da pizza (parte 5).
9	Opcional: abrir `http://127.0.0.1:5000/api/history` pra mostrar o histórico salvo no SQLite.

Pra desligar o site: `Ctrl+C` no terminal.

PARTE 8

Revisão de 5 minutos antes de entrar

- Limpa → Conta → Compara → Vota → (Desiste)
- Limpeza: Meu Amigo Pedro Toma Sorvete Sempre (minúsculas, acentos, pontuação, tokens, stopwords, stem RSLP).
- Bag of words = lista de presença, sem ordem. TF-IDF = palavra rara vale mais. idf = log(N/df).
- Cosseno = ângulo entre vetores, de 0 a 1. Ignora o tamanho da frase.
- Top 5 votam, ganha a maioria (KNN). Empate: soma dos cossenos.
- Abaixo de 0,15 ou palavra desconhecida: fallback, não inventa.
- 5 categorias (Hoje Minha Bibi Saiu Nervosa), 15 intenções, 77 frases, 96 palavras, 12/12 nos testes.
- SQLite num arquivo só. Flask pro site. Conteúdo no `faq_seed.json`.
- Pizza: "quero" era rara, pesou demais, virou stopword.
- Limitações: ordem, sinônimos, palavra rara, sem memória. Melhoria: embeddings (Word2Vec).

Baseado no código do projeto e no material "Representação Matemática de Texto Utilizada em PLN" do Prof. Fabrício Galende Marques de Carvalho.